Codebase Architecture Guide

This document outlines the architecture and organization of the GRPO reinforcement learning pipeline, optimized for Modal GPU environments.

Core Execution

The repository is built to seamlessly transition from local debugging to distributed parallel execution on Modal GPUs.

- `modal_train.py`: The primary entry point for Modal execution. This script orchestrates container builds, sets up volume mounts, and coordinates the pipeline module over a swept configuration matrix.
- `train_grpo.py`: Core TRL training script using Group Relative Policy Optimization. Contains the Hugging Face `SFTTrainer`/`GRPOTrainer` hooks.
- `eval_grpo.py`: Standardized evaluation pipeline for post-training inference benchmarks.
- `reward_fns.py`: Implements the distinct reinforcement learning reward algorithms (e.g., format matching, mathematical accuracy).

Pipeline Orchestration (`pipeline/`)

The `pipeline` module abstracts over the complex matrix executions previously bound inside large monolithic scripts.

- `pipeline/runner.py`: Handles child process coordination, passing appropriate hardware arguments (like Accelerate configurations) down to `train_grpo.py` and `eval_grpo.py`. It is invoked natively by `modal_train.py` or through the wrapper `run_prelim.py`.
- `pipeline/config.py`: Exposes a robust dataclass and YAML merger for configuring models, datasets, bounds, and hardware overrides.
- `pipeline/reporter.py`: Consolidates leaderboards, metrics, and comparisons between baseline models and tuned variants. It generates Markdown and CSV artifacts for immediate observation.

Visual Analytics (`ui/`)

The Streamlit interface allows users to review the sweeping output and interactively test trained models.

- `ui/app.py`: The frontend UI, abstracting Streamlit layouts.
- `ui/core.py`: The backend logic module covering HF model loading, inference logic, and historical `run_metadata.json` / leaderboard loading.

Supporting Subsystems

- `configs/`: Houses YAML definition files dictating training hyper-parameters and pipeline models. The `sweeps/` sub-directory isolates hyper-parameter permutations.

- `scripts/`: Utilities for the platform, notably `healthcheck.py` to ensure PyTorch, CUDA, and environment compatibility before launching deep training loops.
- `archive/`: Historical environment deployments (like Runpod scripts and guides) kept for lineage.